

B Academy
RO

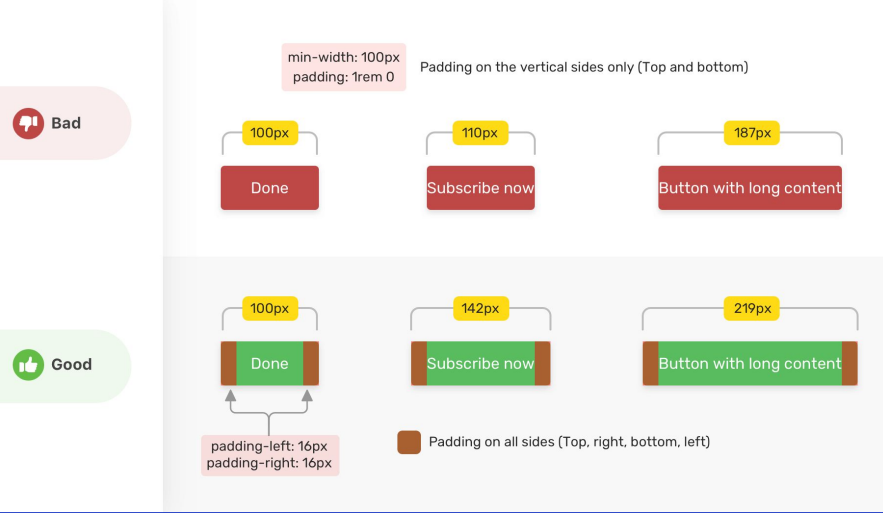


Manual

sizes and spaces

min/max
width/height

min-width



When setting the value of `min-width`, its benefit lies in preventing the used value for `width` property from becoming less than the specified value for `min-width`. Note that the default value for `min-width` is `auto`, which resolves to 0.

We have a button with a varying text within in. The text might range from one word to multiple. To guarantee that it will have a minimum width even if it has one word only, `min-width` should be used. The minimum width is 100px so that even if the button has a very short content, like "Done" word, or an icon only, it will still be big enough to be noticed. This is very important when working with multilingual websites like Arabic.

Min Width and Padding

While `min-width` can extend the button width in case its content is longer, padding on the horizontal sides should be added to achieve a proper looking button.

Example

```
min-width: 300px;  
min-width: 50%;
```



Bad



Good

max-width: 100%



max-width

When setting the value of max-width, its benefit lies in preventing the used value for width property from becoming more than the specified value for max-width. The default value for max-width is none.

A common and simple use case for max-width is using it with images.

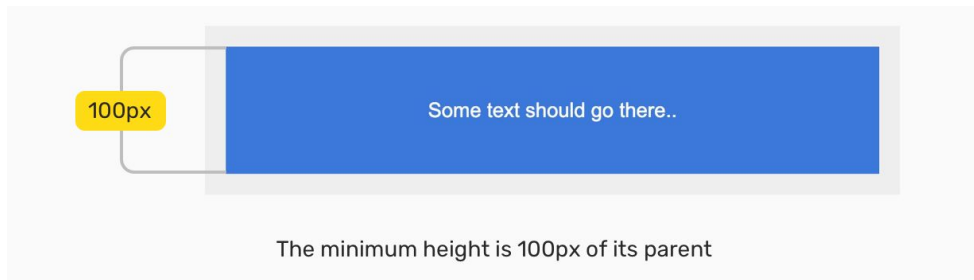
The image is larger than its parent element. By using max-width: 100%, the width of the image won't exceed the width of its parent. In case the image is smaller than its parent, max-width: 100% won't have an actual effect on the image, because it's smaller than its parent.

Example

```
max-width: 300px;  
max-width: 50%;
```

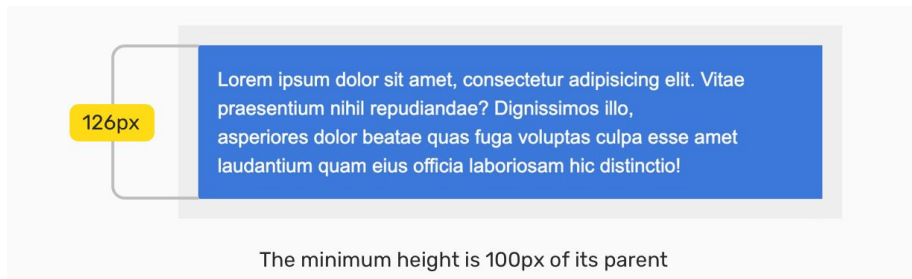
When setting the value of min-height, its benefit lies in preventing the used value for height property from becoming less than the specified value for min-height. Note that the default value for min-height is auto, which resolves to 0.

We have a section with a description text. The goal is to have a minimum height for the section, so it can handle short or long content.



The minimum height is 100px, and with flexbox, the content is centered horizontally and vertically. What would happen if the content were longer? For example, a paragraph?

The height of the section will expand to contain the new content. By having that, we can build components that are fluid and responsive to its content.



min-height

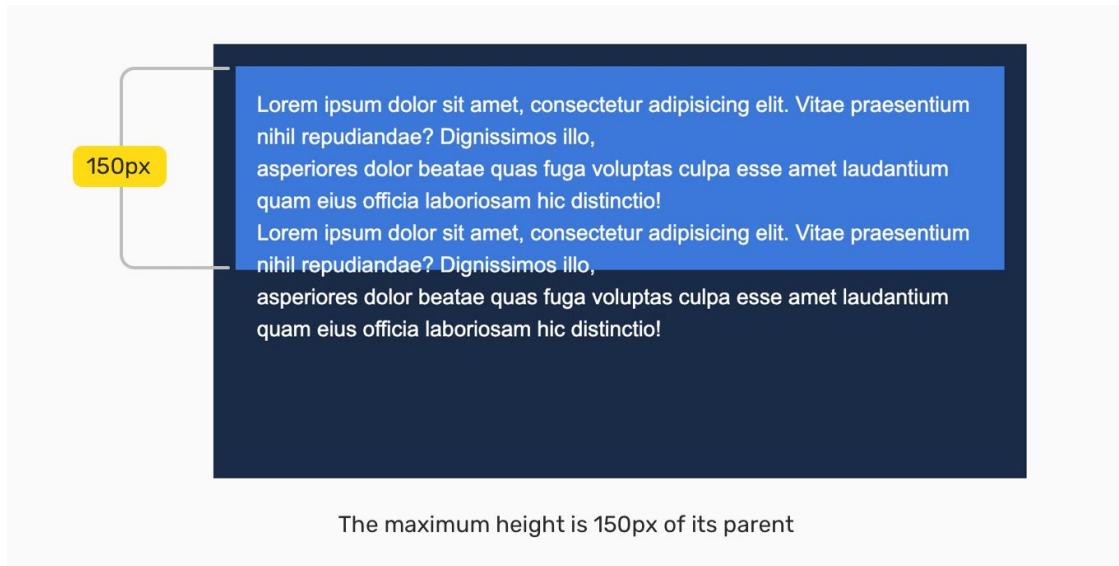
Example

```
min-height: 300px;  
min-height: 50%;
```

When setting the value of `max-height`, its benefit lies in preventing the used value for height property from becoming more than the specified value for `max-height`. Note that the default value for `max-height` is none.

Consider the below example where I set `max-height` for the content. But, because it's bigger than the specified space, there is an overflow. The text is out of its parent boundaries due to that.

max-height



Example

```
max-height: 300px;  
max-height: 50%;
```

Don't use height

Use only min-height

High quality

Lorem Ipsum is simply dummy text of the printing and Ipsum typesetting industry.

High quality

Lorem Ipsum is simply dummy text of the printing and Ipsum typesetting industry. Lorem Ipsum is simply dummy text of the printing and Ipsum typesetting industry.

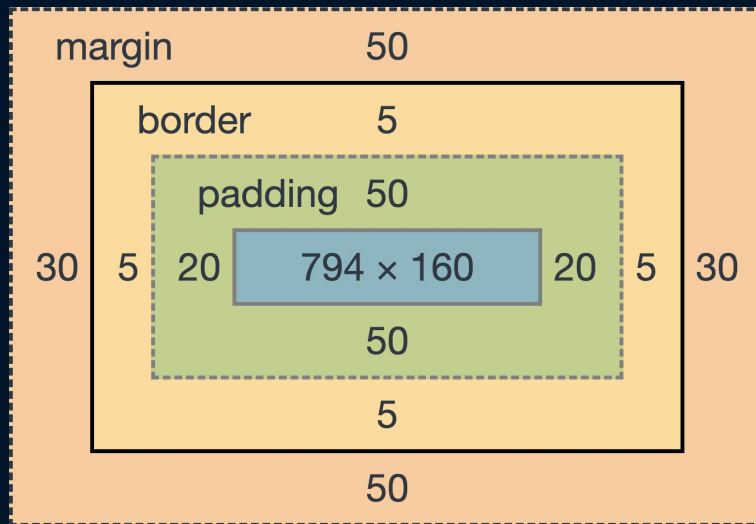
or don't use height

BOX MODEL

Box model

– is essentially a box that wraps around every HTML element.

- **content**
- **padding** – The space inside the element, adding space within the element
- **border**
- **margin** – The space outside the element, pushing neighboring elements away.



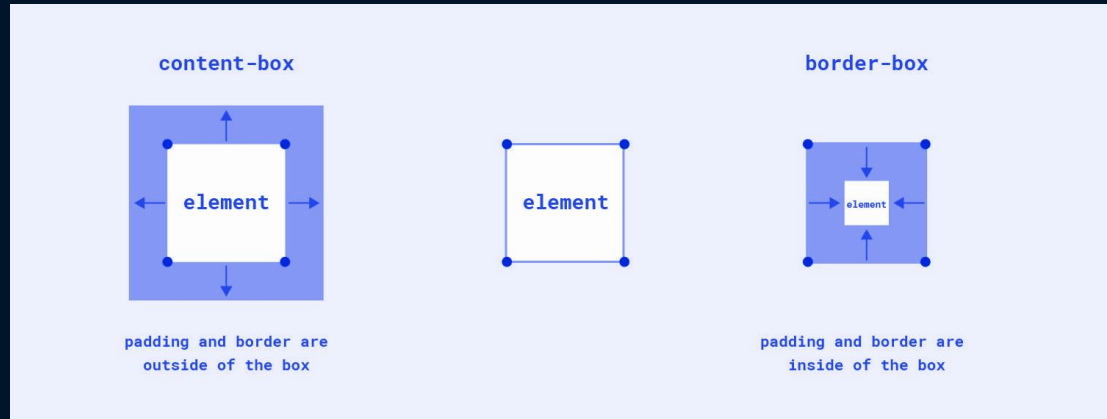
box-sizing property

controls how the box model is handled for the element it applies to.

Default:

content-box – width and height properties include the **content**, but **does not include** the **padding**, **border**, or **margin**.

border-box – width and height properties **include** the **content**, **padding**, and **border**, but **do not include** the **margin**.



The padding CSS property is commonly used in CSS to add space in the inner side of an element.

Remember:

- margin adds space outside an element border
- padding adds space inside an element border

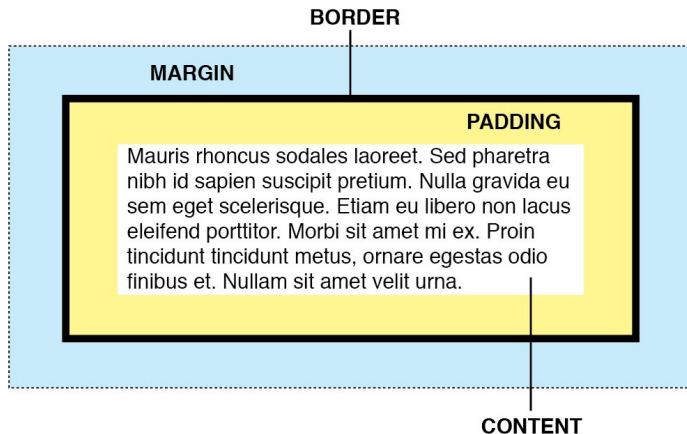
padding

padding has 4 related properties that alter the padding of a single edge at once:

- **padding-top**
- **padding-right**
- **padding-bottom**
- **padding-left**

Units of measurement:

- **px**
- **%**



Example

```
padding-left: 30px;  
padding-right: 3em;
```

padding is a shorthand to specify multiple padding values at the same time, and depending on the number of values entered, it behaves differently:

1 value

Using a single value applies that to **all** the paddings:

top, right, bottom, left.

padding: 30px;

2 values

the first to **bottom & top**,

and the second to **left & right.**

padding: 10px 40px;

3 values

the first to **top**,

the second to **left & right**,

the third to **bottom.**

padding: 10px 80px 5px;

4 values

the first to **top**,

the second to **right**,

the third to **bottom**,

the fourth to **left.**

padding: 40px 15px 5px 30px;

padding shorthands

This paragraph has a padding of 30px on all sides.

This paragraph has a padding of 10px at the top/bottom and 40px at the left/right sides.

This paragraph has a padding of 10px at the top, 80px at the left/right, and 5px at the bottom.

This paragraph has a padding of 40px at the top, 15px at the right, 5px at the bottom, and 30px at the left.

The margin CSS property is commonly used in CSS to add space around an element.

Remember:

- margin adds space outside an element border
- padding adds space inside an element border

margin

margin has 4 related properties that alter the margin of a single edge at once:

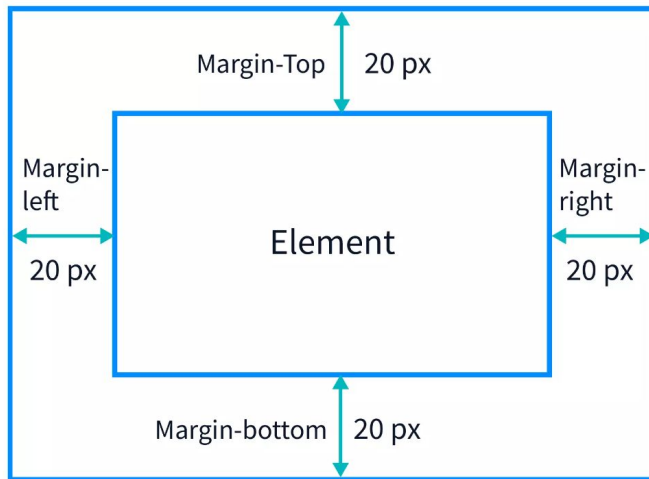
- **margin-top**
- **margin-right**
- **margin-bottom**
- **margin-left**

Units of measurement:

- **px**
- **%**
- **auto**

Can have negative values.

margin-top and **margin-bottom** do not work on inline-boxes



Example

```
margin-left: 30px;  
margin-right: 3em;
```

margin is a shorthand to specify multiple margins at the same time, and depending on the number of values entered, it behaves differently:

1 value

Using a single value applies that to **all** the paddings:

top, right, bottom, left.

margin: 50px;

2 values

the first to **bottom & top**,

and the second to **left & right**.

margin: 15px 100px;

3 values

the first to **top**,

the second to **left & right**,

the third to **bottom**.

margin: 15px 100px 50px;

4 values

the first to **top**,

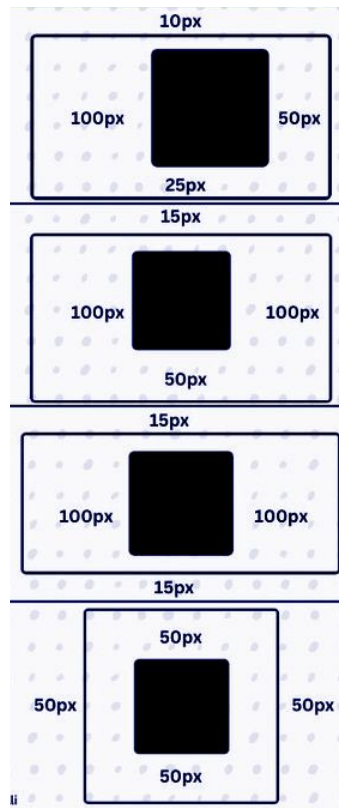
the second to **right**,

the third to **bottom**,

the fourth to **left**.

margin: 10px 50px 25px 100px;

margin



A



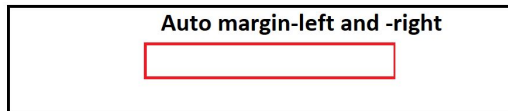
B



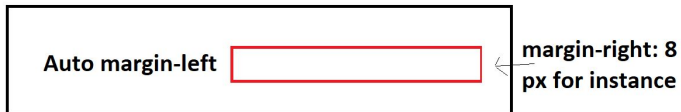
C



D



E



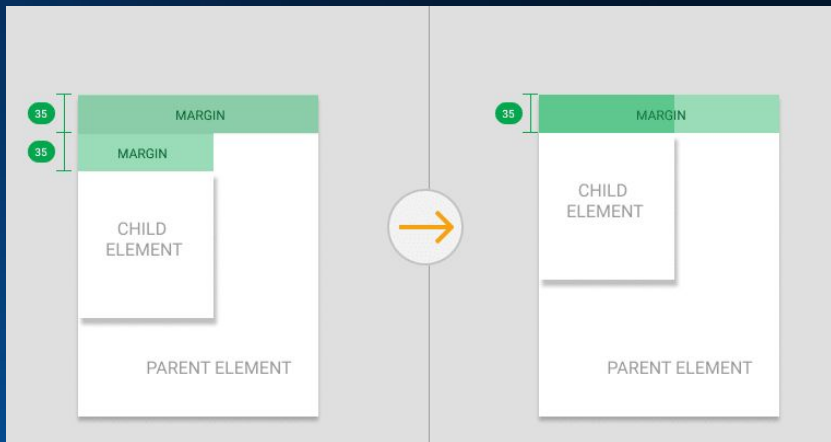
margin: auto 

margin: auto;
margin-right: auto;
margin: 0 auto;

The width of the margin is automatically calculated by the browser and occupies all available space between elements and margins within the container.

Collapsing Margins

- The concept of margin collapsing comes from graphic design. There, you have margins to title and subtitles, but when a subtitle comes just after the title you should not double the margins. This is why they developed the concept of collapsed margins and this is why it happens just on vertical margins.
- **When?**
If there is one thing to remember when you leave this page is that collapsing only happens on vertical margins.
- In elements with **position: absolute** and **position: fixed**, **display: flex**, **display: grid**; **float** margins do not collapse or cause margin drop.

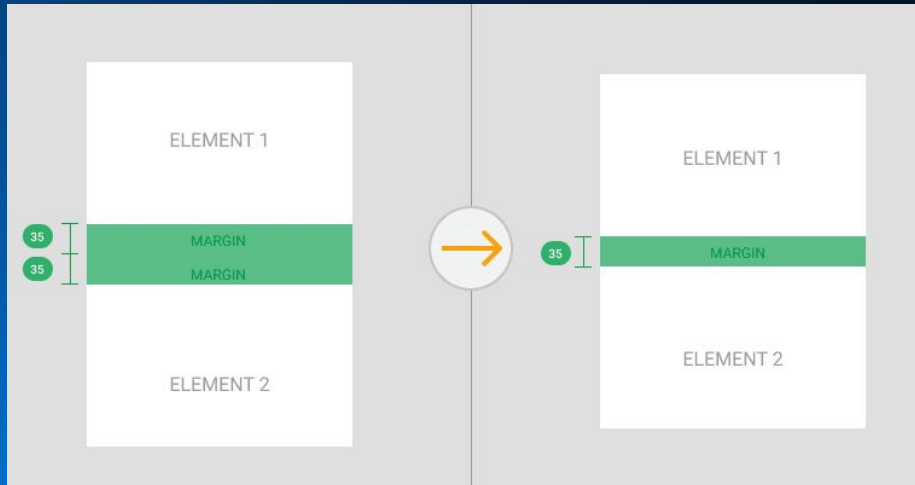


Collapsing Margins

- If **no border, padding, inline part** or clearance separate the margin-top of a parent to his first child, then it is going to collapse.
- The same happens with the last child of the parent.
- Works for the **margin-top** of the first child inside a parent and for the **margin-bottom** of the last child inside a parent.
- The child's **margin** becomes the parent's **margin** or the larger of the two applies.

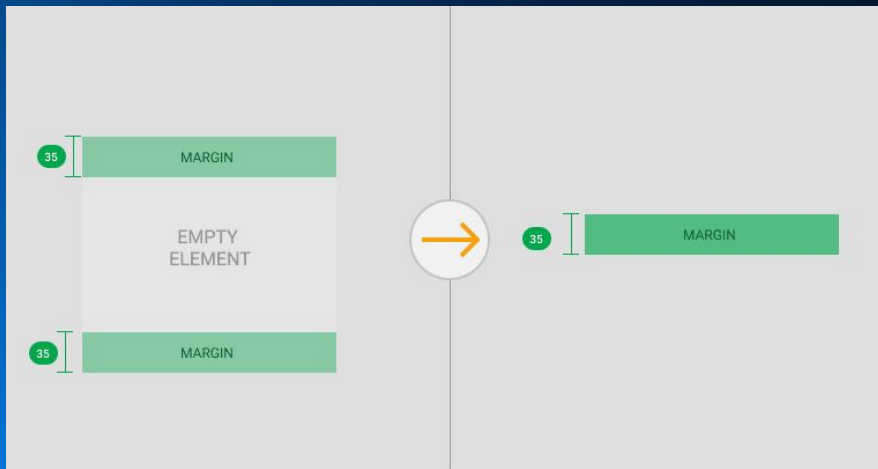
Collapsing Margins: Adjacent Siblings

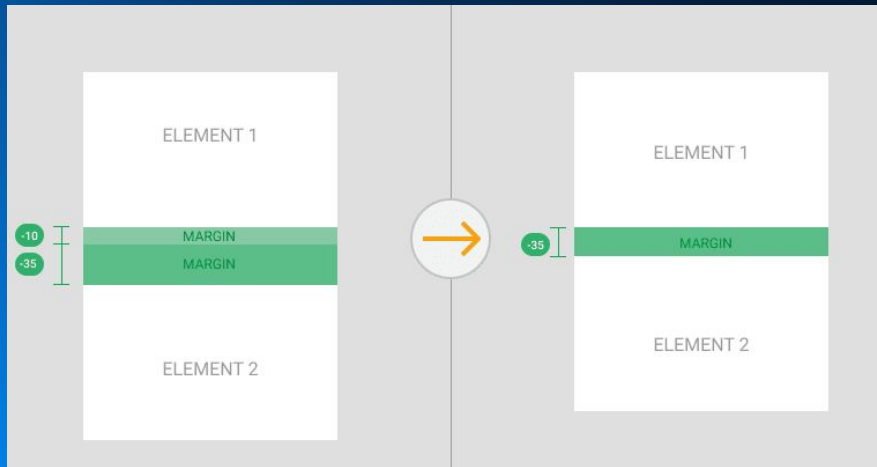
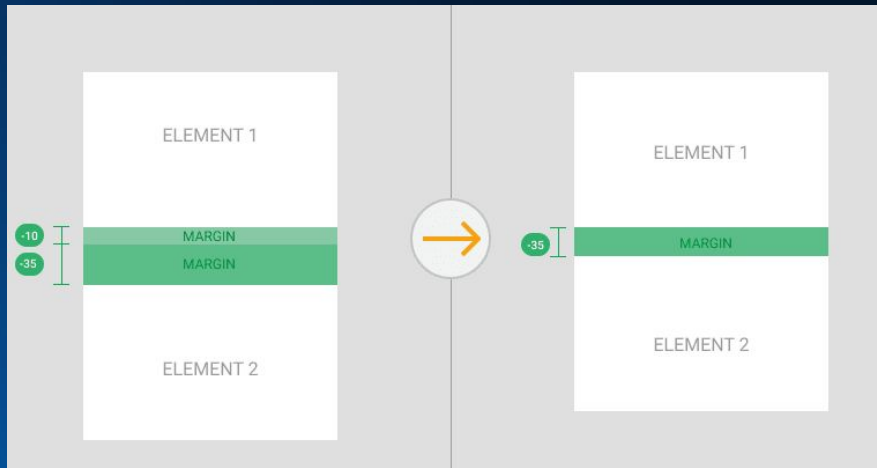
- When there are adjacent siblings, the margin-bottom of the top element collapses with the margin-top of the bottom element. The resulting size is the biggest between the two.



Collapsing Margins: Empty Blocks

- The margin-top and margin-bottom of an empty block collapse if there are no padding, border, inline content avoiding it. Also, it does not collapse if the block does not have a height or min-height attribute.





Collapsing Margins: Negative Margins

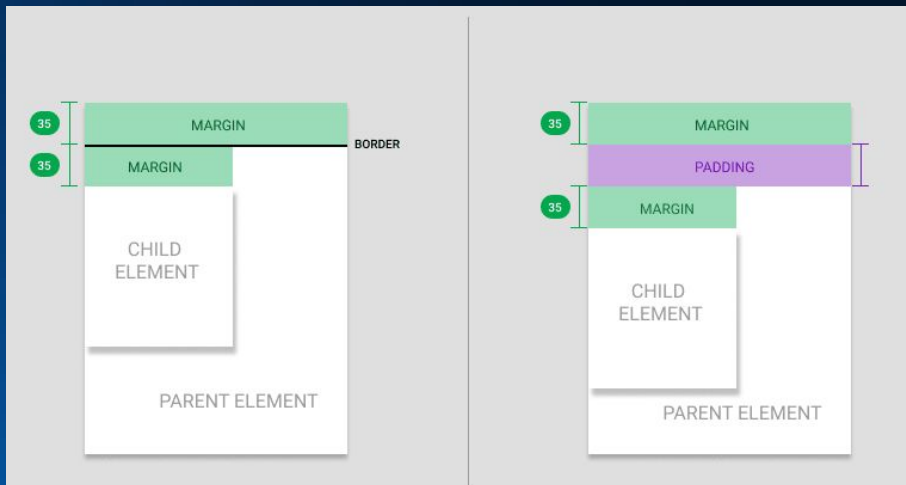
When it comes to negative margins, the resulting size depends on the signals involved.

Negative + Positive

The resulting size of a negative margin and a positive margin is their algebraic sum.

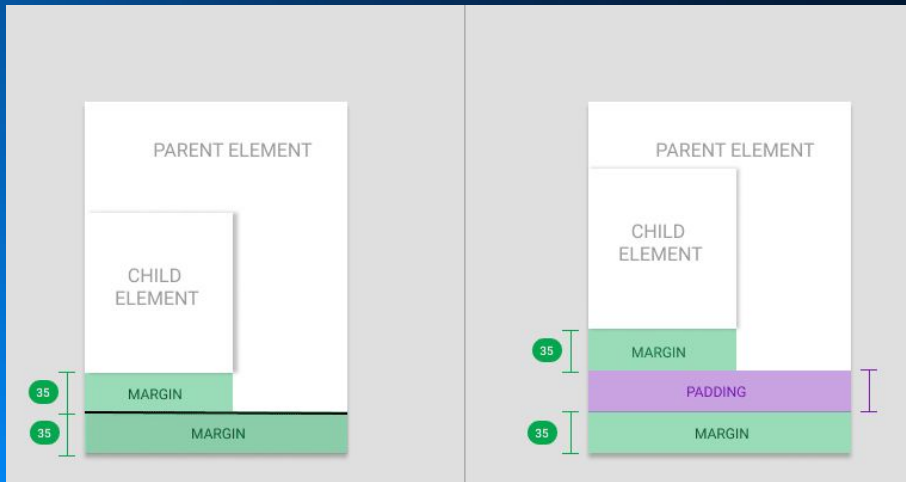
Negative + Negative

When both margins are negative, the resulting size is similar when there are positive + positive. In this, the most negative one wins.



Collapsing Margins: Border or Padding Property on the Parent Element

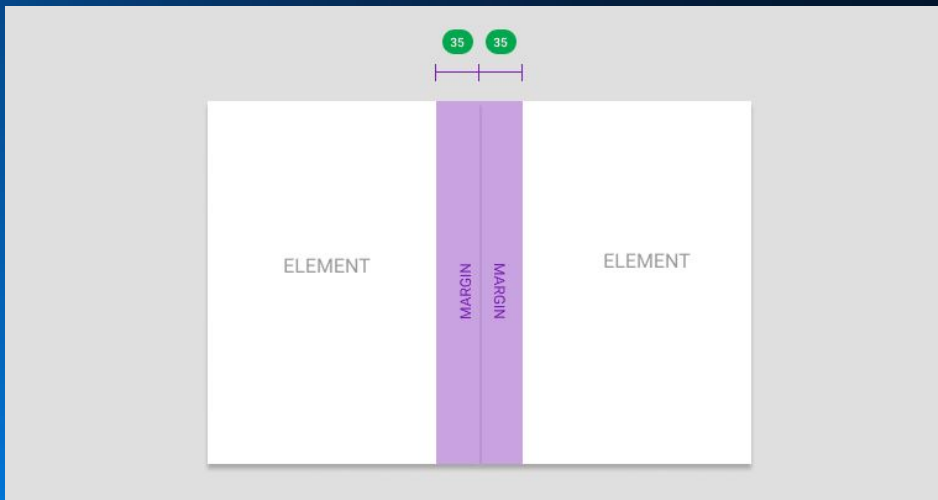
If there is a **border-top** or **padding-top** between the parent and the first-child.



The same happens if there is a **border-bottom** or **padding-bottom** between the parent and its last-child.

Collapsing Margins: horizontal margins

The horizontal margins never collapse.



Do not apply **margin** to BEM blocks

The block shouldn't influence its environment, meaning you shouldn't set the external geometry (margin) or positioning for the block.

logical properties

The CSS logical properties and values module defines logical properties and values that can control layout through logical rather than physical direction and dimension mappings. Logical properties define direction-relative equivalents to their corresponding physical properties. This means content translated into languages with different writing modes will be rendered as intended.

The start of a line is not always the left side of a line. Different writing systems operate in various directions.

- English and Portuguese are written from left to right with new lines added below the previous ones.
- Hebrew and Arabic are right-to-left languages with new lines again being added below the previous ones.
- In some writing modes, the text lines are vertical, written from top to bottom. Chinese, Vietnamese, Korean, and Japanese are traditionally written vertically, from top to bottom, with each new vertical line added to the left of the previous one.
- Traditional Mongolian is also a top-to-bottom language, but new lines are to the right of previous ones.

The logical properties defined in this module enable defining properties relative to the content's writing direction, rather than a physical direction. This means content translated into languages with different writing modes will be rendered as intended.

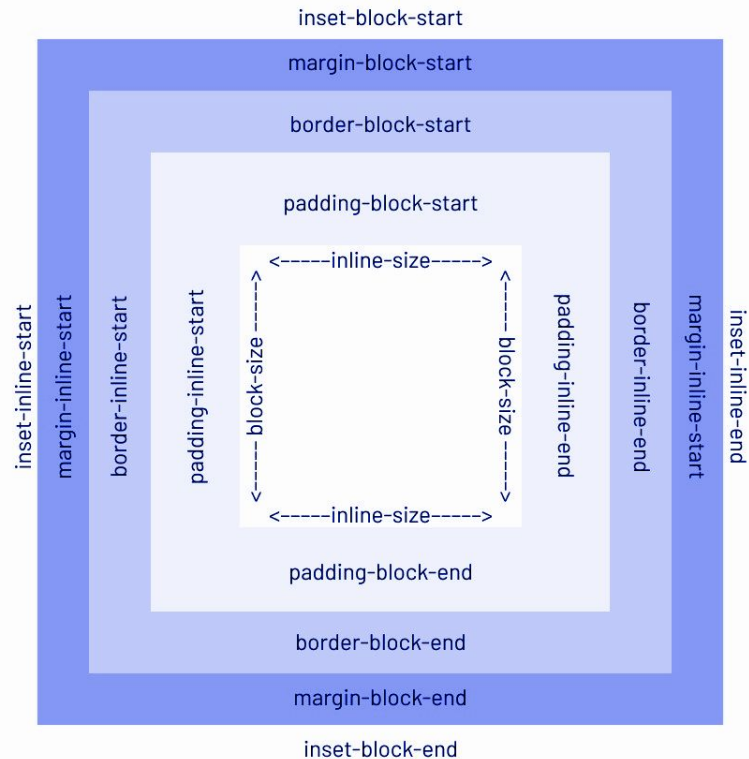
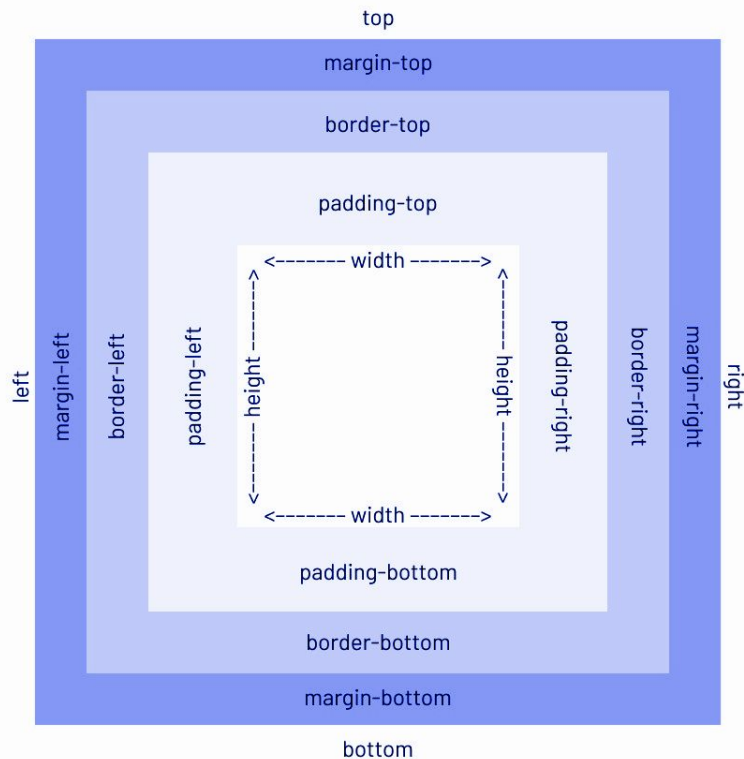
The block dimension is perpendicular to the flow of text within a line, i.e., the vertical dimension in horizontal writing modes, and the horizontal dimension in vertical writing modes. For standard English text, it is the vertical dimension.

The inline dimension is parallel to the flow of text within a line, i.e., the horizontal dimension in horizontal writing modes, and the vertical dimension in vertical writing modes. For standard English text, it is the horizontal dimension.

physical

/

logical



logical properties shorthands

Vertical

Using a single or double value applies that to **vertical** paddings/margins:

top + bottom

`padding-block: 30px;`

`margin-block: 30px 10px;`

Horizontal

Using a single or double value applies that to **horizontal** paddings/margins:

left + right.

`margin-inline: 10px;`

`padding-inline: 10px 40px;`

Centering Content, Full-Width Background

The background of the website should always stretch across the full width of the page.

The content has a width restriction (set by width/max-width) as per the design layout and is centered on the page.

```
<section>
  <div class="content-wrapper">

section { background-color: grey; }

.content-wrapper {
  width: 1400px;
  margin: 0 auto;
}
```

Demo

